Notification System

Overview

Notifications are delivered through three channels:

- in-app persistence in MongoDB
- realtime Socket.IO emission to the recipient’s user room
- push delivery through OneSignal

The system supports social, moderation, reward, and community-post notifications with basic user preference filtering.

Current implementation

Core service

- `services/notification.service.js`
- validates required fields
- skips self-notifications
- loads receiver preferences
- creates a `Notification` document
- emits `notificationCreated` to the receiver room
- optionally sends push through OneSignal

API layer

- `routes/notifications.routes.js`
- supports notification creation and notification retrieval for the logged-in user
- auto-resolves target types for common events such as likes, comments, follows, and community posts

Community notifications

- `services/communityPostNotification.service.js`
- fetches eligible community members
- skips blocked, muted, suspended, and self targets
- batches delivery in groups of 25

Important modules

- `services/notification.service.js`
- `routes/notifications.routes.js`
- `utils/onesignal.js`
- `models/notifications.model.js`

Known issues

- target URL logic is duplicated in both the route layer and the notification service
- delivery policy is synchronous from the request path for many event types
- OneSignal configuration is flexible but operationally noisy because multiple env names are accepted

Scaling concerns

- community post fan-out performs repeated recipient checks and per-recipient notification writes
- push sends are still coupled to request-time flows in many cases
- no message queue isolates external push latency from user-facing API latency

Recommended improvements

1. centralize target resolution and notification formatting in one module
2. move high-fan-out delivery to queue-backed workers
3. standardize notification type enums and payload schemas
4. add delivery metrics and dead-letter handling for failed push sends